UL2IN002-2026fev
Introduction à la programmation objet

Projet 2026fev – Thème : "la Mer"

Présentation. Ce projet a pour but la réalisation d'un programme mettant en oeuvre la plupart des points du cours vus ce semestre. Il représente un travail devant prendre environ 5 à 6 heures. Il n'y aura pas de séance de TP dédiée au travail sur ce projet : vous devrez y travailler en dehors des heures des séances. Ce projet est à réaliser seul ou en binôme.

Le travail demandé est de proposer et d'implémenter un programme qui réalise une simulation se déroulant sur un terrain représenté sous la forme d'un tableau à 2 dimensions contenant des ressources et sur lequel des agents peuvent se déplacer et agir.

Thème. Le thème à respecter pour votre programme est "la Mer". Tout programme sur ce thème est donc réalisable. Dans ce thème, le "terrain" peut représenter la mer.

Par exemple, cela peut être la simulation de l'évolution de ressources poissonnières : des bateaux de pêche (les agents) se déplacent dans la mer et pêchent des poissons comestibles (une ressource) et d'autres choses non comestibles (d'autres ressources) qui sont rejetés ; les ressources évoluent au cours du temps et des stastistiques sont réalisées.

Autres exemples : surveillance de l'évolution du phytoplancton ou des coraux ou des baleines ou des plastiques ou autres... dans la mer en fonction de facteurs étudiés (température de l'eau, agents pollueurs ou dépollueurs...) ; îles, bateaux pirates et recherche de trésors ; jeu de survie sur une île ; jeu qui mixe bataille navale et démineur ; monde marin ou sous-marin imaginaire...

La simulation doit avoir un objectif. Exemples : faire des statistiques sur la quantité de poissons récoltés ou sur le nombre de baleines observées ; essayer de trouver le plus de pièces d'or possible en fonction des ressources disponibles pour avancer et malgré les bateaux pirates et les pièges...

Cahier des charges. Le programme à réaliser doit respecter le cahier des charges suivant :

- la simulation se déroule sur un terrain représenté à l'aide de la classe `Terrain` qui est fournie. La classe `Terrain` est finale.
- la simulation gére :
  - des `Ressource` (classe qui est aussi fournie) placées sur le terrain, celles-ci ne peuvent pas se déplacer d'elles-mêmes mais elles peuvent être retirées du terrain,
  - des agents qui ont la capacité de se déplacer et de réaliser des actions sur le terrain (par exemple, prendre des ressources ou en déposer). Remarque : contrairement aux ressources, les agents ne sont pas mémorisés sur le terrain.
- votre simulation devra utiliser au moins 2 types de ressources :
  - des ressources dont l'état interne ne change pas (leur quantité reste constante, sauf si cette quantité est modifiée par un agent)
  - des ressources dont l'état interne peut évoluer sans l'intervention d'un agent (par exemple, dont la quantité peut évoluer au fil du temps...)

Feuille de route.

1. Choisir le thème de la simulation, identifier les agents et les ressources, réfléchir au but de la simulation et aux interactions entre les agents, les ressources et le terrain (ce qui en fait une simulation). Noter que vous pouvez ajouter d'autres classes (et interfaces) en plus des classes agents et des classes de ressources.

2. Le terrain est représenté par un tableau à 2 dimensions de `Ressource`. Une ressource est caractérisée par un type et une quantité. Comprendre comment utiliser les classes `Terrain` et `Ressource` en étudiant la documentation javadoc fournie et en expérimentant avec la classe `TestTerrain` aussi fournie. Voir la liste des fichiers fournis dans l'Annexe à la fin.
3. Écrire une classe pour représenter un agent (le nom de la classe n'est pas forcément "Agent"). Les agents ne sont pas mémorisés sur le terrain, mais les agents connaissent (dés leur création) le terrain sur lequel ils évoluent. Un agent possède aussi deux attributs qui correspondent à sa position sur le terrain (numéros de ligne et de colonne). Cette classe doit contenir les deux méthodes suivantes :
   - `distance(lig,col)` qui rend la distance (euclidienne ou autre) séparant l'agent de la case de coordonnées `(lig,col)` où `lig` est un numéro de ligne et `col` un numéro de colonne ;
   - `seDeplacer(lig,col)` qui change la position de l'agent sur le terrain et le place en `(lig,col)`.

   Vous pouvez également rajouter dans cette classe tous les attributs, méthodes et méthodes abstraites que vous jugerez utiles pour votre simulation, ainsi que des classes descendantes.
4. Écrire la classe `Simulation` qui contient un terrain, une liste ou un tableau d'agents, et une liste ou un tableau de ressources. Cette classe possède un constructeur qui, par exemple, place aléatoirement `m` ressources sur le terrain et génère `n` agents. Vous pouvez ajouter dans cette classe la gestion des règles de votre simulation. Cette classe doit réaliser le travail suivant :
   (a) initialiser le terrain avec des ressources ;
   (b) réaliser une étape de la simulation :
      - chaque agent réalise une action,
      - mise à jour de chaque case du terrain,
      - affichage des informations sur ce qui s'est produit durant l'étape ;

   (c) recommencer (b) jusqu'à un nombre choisi d'étapes ou jusqu'à ce qu'une condition soit vérifiée.
5. Écrire la classe `TestSimulation` contenant un `main`, dans laquelle des essais de simulation sont effectués (par exemple, en variant le nombre de ressources et le nombre d'agents), et qui produisent des logs dans le terminal (ou dans un fichier). Ces logs doivent correspondre aux différentes actions réalisées et à leurs résultats. Remarque : de tels logs seront à rendre dans le compte-rendu.

Vous pouvez ajouter d'autres attributs et méthodes aux classes demandées. Vous pouvez aussi proposer d'autres classes et des interfaces. Cependant, le programme doit respecter les contraintes suivantes :

- entre 6 et 12 classes (ou interfaces) : privilégier la qualité (choix des attributs et des méthodes de chaque classe), plutôt que la quantité des classes. Remarque : les classes `Ressource`, `Terrain` et `TestTerrain` ne comptent pas dans le comptage du nombre de classes/interfaces.
- au moins une hiérarchie d'héritage à 3 niveaux.

Modalités et soumission du projet. Ce projet est à réaliser seul ou en binôme (2 étudiants maximum) d'un même groupe de TD. Remarque : dans le cas d'un travail en binôme, une seule remise sera faite sur un des 2 comptes Moodle.

Vous devez soumettre sous Moodle **un seul fichier** archive (zip, tar, tgz) contenant un répertoire portant votre nom (et le nom votre binôme). Ce répertoire contiendra les fichiers et répertoires suivants :

- un fichier PDF généré à partir du fichier `description.odt` complété avec les informations demandées. Toutes les classes écrites dans le projet doivent être copier/coller à la fin du fichier.
- un répertoire `src/` contenant tous les fichiers `.java` écrits pour ce projet. **Attention :** tous les fichiers `.java` doivent contenir au début votre nom et le nom de votre binôme.
- un répertoire `doc/` contenant la javadoc générée pour toutes vos classes.
- un fichier `logs.txt` donnant 2 ou 3 exemples de résultats des simulations réalisées.

Soutenance. La soutenance du projet aura lieu lors de la dernière séance de TP du groupe.
Déroulement de la soutenance :

- **2 minutes :** présentation par les étudiants de leur projet. Merci de vous préparer pour présenter votre projet en moins de 2 minutes. Vous devez montrer au moins une simulation. Essayer de mettre en valeur ce que vous avez fait. Si vous êtes en binôme, les deux doivent parler.
- **5 minutes :** questions de l'enseignant pour vérifier que c'est bien vous (et votre binôme) qui avait fait ce projet, et que les 2 membres du binôme ont travaillé.

Attention : la soutenance est obligatoire. Un projet rendu mais pour lequel il n'y a pas de soutenance sera noté 0, car l'un des buts de la soutenance est de vérifier que :

- c'est bien vous (et votre binôme) qui avez fait le projet,
- et que les 2 membres du binôme ont travaillé.

Remarques :

- Il n'est pas interdit d'utiliser une IA pour générer du code dans votre projet, mais vous devez être capable d'expliquer pendant la soutenance tout code généré par IA.
- De plus, il se pourrait que une ou des questions de l'examen soit directement liées aux projets, il faut donc que les 2 membres du binôme aient compris toutes les parties du projet.

Barème indicatif. La note finale (sur 20) sera déterminée de la façon suivante :

8 points	Mise en application des points vus en cours (voir le tableau à remplir dans le fichier `description.odt`)
5 points	Diagramme UML fournisseur complet et correct Qualité de la conception (comment les classes, interfaces et leurs liens sont justifiés) Respect des bonnes pratiques de POO Respect des contraintes (hiérarchie d'héritage à 3 niveaux, nombre de classes)
3 points	Clarté et lisibilité du programme (noms des variables, méthodes, classes, présence de commentaires explicatifs) Génération de la javadoc Programme qui réalise des simulations (générations et qualités des logs)
4 points	Soutenance : présentation d'une démonstration des fonctionnalités du programme, réponses aux questions

Annexe : fichiers fournis. L'archive `UL2IN002-projet2026fev.tgz` contient :

- le fichier `description.odt` que vous devez remplir avec les informations de votre projet,
- le fichier `Terrain.class`,
- le fichier `Ressource.class`,
- le fichier `TestTerrain.java` qui fournit un exemple d'utilisation des classes `Terrain` et `Ressource`,
- le répertoire `doc/` qui fournit la javadoc des classes `Terrain` et `Ressource`. Pour visualiser cette javadoc, il faut ouvrir le fichier `index.html` avec un navigateur internet.

Les classes `Terrain` et `Ressource` ne doivent pas être modifiées, c'est pour cela qu'elles ne sont fournies que sous forme compilée (format `.class`). Pour utiliser les 2 classes `Terrain` et `Ressource`, vous devez mettre leurs fichiers `.class` dans le même répertoire que vos fichiers `.java`.